

OpenJev: Bounded Decisions, Causal Memory, and Doom Control

OpenJev project

<https://github.com/kw2828/OpenJev>

September 16, 2026

Working research draft; not a conference submission

Abstract

OpenJev exposes request-defined candidate decisions and a reproducible Doom control testbed. An initial Bayesian pilot did not establish a control gain from posterior averaging. A frozen 1,980-episode causal-history ablation improved utility over the original model but did not clear an equally sized current-only control. Three subsequent prospective engineering studies ran 6,408 new episodes. A simple memory of observed ammo decreases or hits improved net utility over rules by 0.823 in Defend the Center and 5.435 in Defend the Line, with identical kills across all 192 paired confirmation episodes. Adding this event memory to the same fitted four-feature ensemble also improved command utility while preserving paired kills. However, all broader continuation gates failed, including historical-model comparisons and, for the selected fitted model, Line kill noninferiority. These results establish a narrow command-efficiency improvement under the stated action cost, not better combat performance, a new Bayesian method, or ICLR novelty. Frozen protocols, failed attempts, traces, figures and a browser-local decision playground are released.

1 Motivation and scope

A bounded application decision often needs a stable identifier and a score rather than generated prose. OpenJev accepts text context, a set of questions, and candidate identifiers with English descriptions. The application maps the returned identifier to an action. A pretrained Qwen language backend scores next-token candidate labels; it remains an autoregressive transformer and is not a new architecture. Questions are processed sequentially, without shared-prefix optimization.

The project is inspired by the typed-decision interface of Jev. It does not reproduce Jev’s weights or proprietary Reinforcement Learning for Calibrated Decisions (RLCD). The research model in this report is a separate seven-feature Bayesian logistic outcome predictor. There is no language-model RL training, PPO, GRPO, or Bellman update in the pilot.

The initial pilot asks: *does approximate Bayesian averaging or a conservative posterior bound improve firing decisions relative to the point estimate of the same learned model?* The original language-controller recording is a demonstration, not one of the fitted models or a baseline in this experiment.

2 Decision interface and outcome model

For candidate logits $z_1, \dots, z_K$, the interface returns the conditional choice distribution

$$q(a_i \mid c, Q, C) = \frac{\exp(z_i)}{\sum_{j=1}^K \exp(z_j)}. \quad (1)$$

Here c is context, Q is the question, and C is its candidate set. Candidate identifiers are preserved by the runtime. These uncalibrated scores sum to one but are not probabilities of task success. The runtime also reports candidate-label mass under the full vocabulary; restricting the denominator to candidates does not eliminate omitted-answer uncertainty.

The outcome model instead predicts a specified binary event when a fire command is issued. Let $x \in \mathbb{R}^7$ contain an intercept, visibility, clipped absolute aim error, target half-width, clipped distance, scaled health, and clipped ammo. With $y = 1$ for a hit-count increase during the following seven game tics,

$$p(y = 1 \mid x, w) = \sigma(x^\top w), \quad w \sim \mathcal{N}(0, I), \quad (2)$$

$$\hat{w} = \arg \min_w \sum_n \left[\log(1 + e^{x_n^\top w}) - y_n x_n^\top w \right] + \frac{1}{2} \|w\|^2, \quad (3)$$

$$\Sigma = \left[I + X^\top \text{diag}(p_n(1 - p_n))X \right]^{-1}. \quad (4)$$

Newton steps with line search fit the MAP estimate, with at most 40 iterations. A Laplace approximation uses $w \mid D \approx \mathcal{N}(\hat{w}, \Sigma)$. Twenty-node Gauss–Hermite quadrature approximates $\bar{p}(x) = \mathbb{E}_{w \mid D}[\sigma(x^\top w)]$. The lower probability used for conservative gating is

$$p_{\text{low}}(x) = \sigma \left(x^\top \hat{w} - 1.645 \sqrt{x^\top \Sigma x} \right). \quad (5)$$

This is approximately a one-sided 95% posterior lower bound conditional on the fitted approximation, not a conformal or frequentist coverage guarantee. The implementation separates $\text{Var}_{w \mid D}(p)$ and $\mathbb{E}_{w \mid D}[p(1 - p)]$. Neither decomposition corrects feature misspecification or temporal dependence.

The three learned policies issue fire when p_{MAP} , $\bar{p}$, or p_{low} respectively exceeds the fixed threshold 0.25. This is a myopic decision rule; it does not account for future survival or information value. A negative Brier reward primitive is available in the package, but is *not* the fitting loss used here.

3 Experimental design and measurement

We use ViZDoom (Kempka et al., 2016), with privileged visible-actor labels and game variables rather than pixels. Five independent collection seeds each produce 16 training episodes in Defend the Center. The behavior policy requests fire with probability 0.6 and randomizes steering with probability 0.2. Only issued fire-command windows provide outcome labels; cooldown windows without a hit count as failures. Waiting does not generate a synthetic negative label.

All five evaluated firing policies share rule-based steering. The comparisons are the existing rule, the tiny imitation model’s firing output, logistic MAP, Bayesian mean, and Bayesian lower bound. Each learned policy is evaluated on ten paired seeds per scenario for each of five training replicates. The rule and imitation baselines are each run once per seed and reused in paired comparisons, not counted as independent copies. Evaluation seeds 40000–40009 are disjoint from fitting. Episodes have at most 180 seven-tic decisions, and there is no evaluation-time fitting or threshold tuning.

The 440 episodes comprise 80 training episodes, 20 common-audit episodes and 340 evaluation episodes. The common audit applies a randomized behavior policy to supply the same observed firing events to all fitted models. It has ten episodes per scenario, containing 349 center and 700 line events. These events are dependent within episodes and do not expose every counterfactual action outcome. Utility is

$$U = \sum_{t: \text{issued fire}} (y_t - 0.25). \quad (6)$$

It measures command-window outcomes, not per-bullet accuracy or the default game reward. Intervals use 10,000 paired crossed-bootstrap resamples over five training replicates and ten evaluation seeds. They are exploratory, unadjusted intervals with small sampling units.

Preserved failed measurement. The preceding v1 run also completed 440 episodes, but used net ammo decrease to detect shots. Ammo replenishment in Defend the Line masked firing, causing zero detected shots and empty calibration audits. The original traces and quality exception are retained. V2 changes the estimand to commanded firing windows and uses fresh evaluation seeds; it is a new development study, not a repaired identical replication or sealed confirmation. Only v2 supports the cross-scenario tables here.

4 Bayesian pilot results

Controller	Center utility	Line utility	Line kills
Rule baseline	3.175	13.900	26.90
Tiny imitation	3.150	13.100	25.40
Logistic MAP	0.855	5.715	16.34
Bayesian mean	0.870	5.140	16.96
Bayesian lower bound	1.000	0.575	4.76

Table 1: Mean v2 outcomes. Learned controllers have 50 evaluation episodes per scenario; each existing baseline has ten. All policies share rule steering. The table is generated directly from the preserved analysis JSON.

Bayesian mean minus MAP utility was $+0.015$ in the center scenario (exploratory 95% interval $[-0.705, 0.725]$), and -0.575 in the line scenario ($[-3.745, 1.960]$). These intervals do not establish a benefit, equivalence, or absence of smaller effects. The conservative lower bound performed worse in the line scenario: -5.140 relative to MAP ($[-9.120, -1.720]$). Strong rule and imitation baselines further limit any claim of learned-control progress.

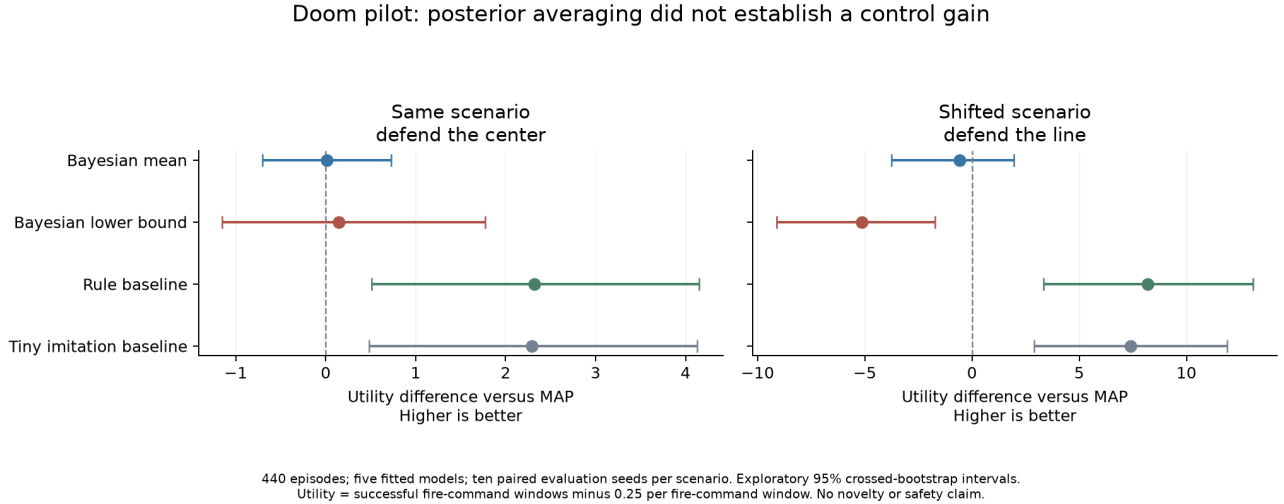


Figure 1: Paired utility differences from MAP. Positive values favor the compared controller. Intervals account for training replicate and episode-seed variation, but are exploratory and unadjusted.

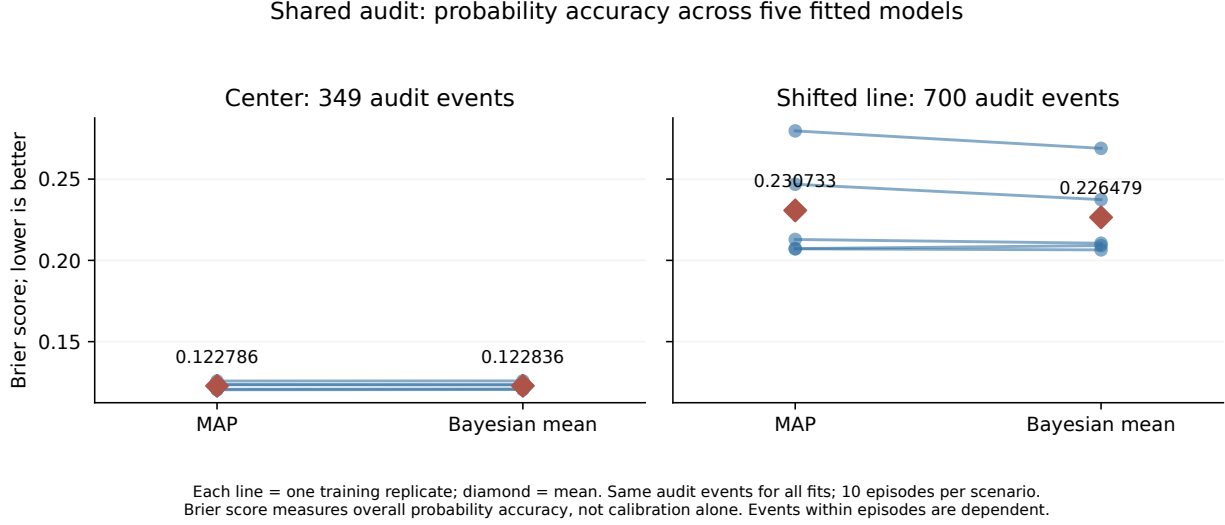


Figure 2: Paired audit Brier scores across five fitted models. The same events are reused across fits, so the five lines are not five independent audit datasets. Red diamonds are means, not confidence intervals.

On the shared line audit, the mean Brier score decreased descriptively from 0.230733 to 0.226479 with Bayesian averaging; in the center it changed from 0.122786 to 0.122836. Brier score measures overall probability accuracy and is not an isolated calibration diagnostic. No uncertainty interval is reported for this descriptive audit comparison. Its apparent improvement in the shifted scenario did not coincide with better mean control utility. This local result motivates measuring prediction and decision performance separately, rather than treating one as a substitute for the other.

5 Frozen causal-memory follow-up

The initial head omitted temporal state. We therefore tested a history extension against both the original seven-feature model and a 17-feature current-only expansion. The protocol and runner were committed at 04ce32d before evaluation. All variants reuse exactly the same 80 historical training episodes and commanded-fire labels. The five original MAP weight vectors were reproduced exactly, ruling out changed fitting data as the source of the comparison.

History appends the previous six non-intercept observation features, previous issued-fire and observed-hit indicators, time since the last issued fire command (capped at ten windows and scaled), and a has-previous indicator. History resets per episode, is updated only after action feedback, and contains no current-window outcome or hidden weapon readiness. The current-only expansion instead appends squares and four interactions. It matches feature count, but does not ensure identical effective capacity or regularization geometry. Both MAP and posterior mean are evaluated for each representation.

The new run uses untouched evaluation seeds 50000–50029 and separate audit seeds 60000–60029 in each scenario. It comprises 1,920 policy evaluations and 60 common-audit episodes; reused training episodes are not counted as new executions. All policies share rule steering, seven-tic windows and a cost of 0.25 per issued fire window. The episode cap remains 180 decisions. Some line episodes hit this cap, so results concern a capped horizon. Evaluation job order is randomized.

Continuation rule. For each scenario, history MAP must improve over both original MAP and expanded MAP by at least 0.5 mean net utility, with a positive lower endpoint of a Bonferroni-adjusted

98.75% interval. Net utility is raw utility minus one unit per second of accumulated decision computation. Four primary contrasts use 20,000 paired crossed-bootstrap draws over five fits and 30 seeds. These are finite-sample exploratory intervals, not a guaranteed test or external confirmation. History must also use no more than three times the original model’s aggregate decision time, with every episode’s p95 decision time below 1 ms.

Timing includes feature construction, steering, prediction, decision construction and history updates. It excludes engine observation/advance and trace I/O. MAP uses a direct dot product; posterior averaging includes quadrature. Lightweight document editing occurred concurrently on the workstation; browser GPU inference began only after this run ended. The timing screen is local, not a controlled systems benchmark.

Policy	Center U	Line U	Center ms	Line ms
Current MAP	0.823	6.493	0.010	0.010
Expanded MAP	1.012	4.845	0.019	0.018
History MAP	2.603	13.795	0.020	0.019
Current Bayes	0.683	5.213	0.023	0.021
Expanded Bayes	1.027	6.205	0.030	0.028
History Bayes	2.837	12.680	0.030	0.030
Rules	2.875	15.233	0.005	0.005
Imitation	2.917	14.817	0.028	0.024

Table 2: New memory-ablation results: raw command-window utility U and mean decision time in milliseconds. Each learned policy has 150 episodes per scenario; each fixed baseline has 30. Rules and imitation remain strong controls.

History MAP improved net utility over original MAP by +1.779 in Center (adjusted interval [0.277, 3.564]) and +7.301 in Line ([0.351, 14.180]). Against expanded MAP, the differences were +1.591 ([−0.349, 3.571]) and +8.950 ([−0.972, 16.998]). Both timing gates passed: history used approximately 1.92 and 1.94 times the original per-decision compute, and maximum episode p95 times were 0.135 and 0.123 ms. The two capacity-control intervals nevertheless include zero. Thus the full continuation gate failed.

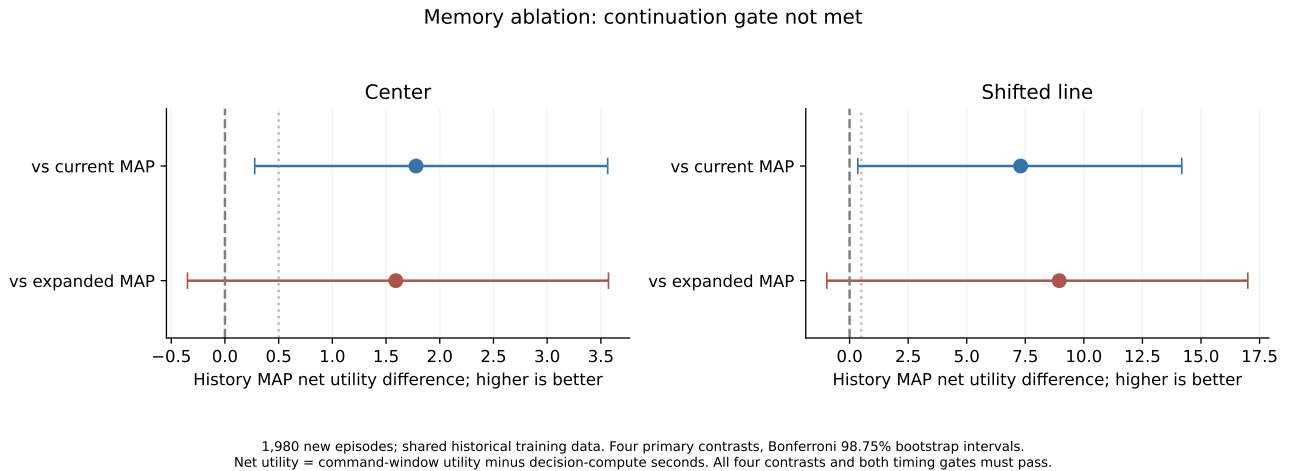


Figure 3: All four predeclared memory contrasts. A positive mean alone is insufficient for continuation; both capacity-control intervals cross zero. The dotted vertical line marks the minimum required mean improvement of 0.5.

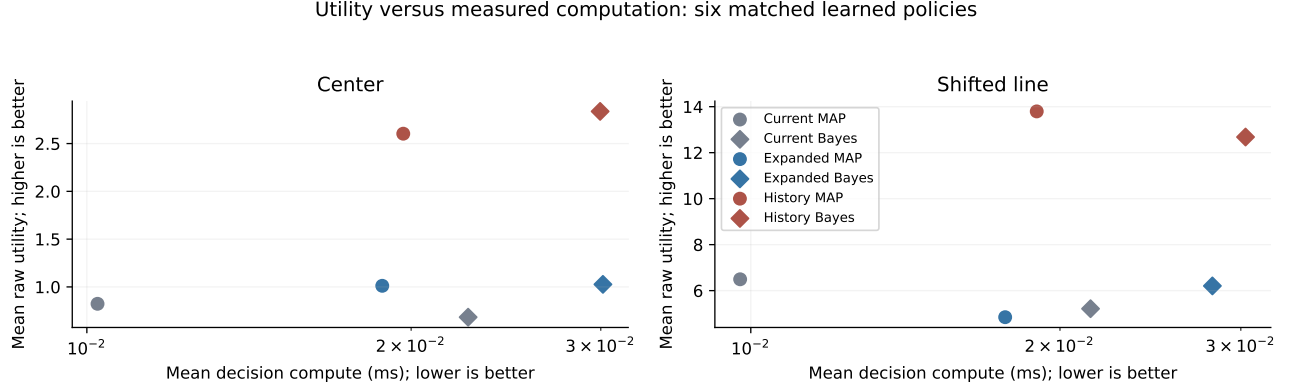


Figure 4: Measured utility and decision compute for the six learned policies. These descriptive points do not establish a statistically dominant frontier. Gate decisions use the paired primary contrasts and timing limits, not visual selection of the best point.

Shared-audit mean MAP Brier scores changed from 0.131446 to 0.119535 in Center and 0.243435 to 0.228333 in Line with history. These descriptive probability-accuracy results do not replace the primary control comparisons. No improvement uniquely attributable to Bayesian inference or learned recurrent computation is established. Adaptive-compute and second-environment efficacy runs were not launched, as required by the frozen failure rule; this original gate was not rerun or relaxed. The separately authorized engineering studies below use new protocols and seeds. The evidence suggests weakness in the original memoryless specification but does not isolate history as the cause relative to the richer current-only representation.

6 Prospective engineering improvements

After the failed memory gate, we initiated separately authorized improvement studies. Each new protocol was committed before its development run, used new development and confirmation seeds, and preserved preceding failures. These are an adaptive sequence of engineering studies, not repetitions of an unchanged sealed test. Interval adjustments apply within each study, not across the entire research sequence.

All new runs retain rule steering, seven-tic decisions, the 180-window cap and the 0.25 cost per issued-fire window. The primary net utility subtracts one unit per second of decision compute, excluding engine observation/advance and trace I/O. Lightweight coding and reporting ran concurrently on the workstation; timing remains a screening measurement. The confirmation rule requires at least 0.5 mean net-utility gain and a positive lower interval endpoint against rules and the five historical history-MAP fits in both scenarios. It additionally requires each kill-difference lower endpoint to exceed -1 , and every selected-controller episode's p95 decision time to remain below 1 ms. Eight primary contrasts use 20,000 paired bootstrap draws and 99.375% percentile intervals; history comparisons resample both seeds and training fits. These finite-sample intervals are exploratory.

6.1 Ammo-only cadence

The first new study, frozen at 5bd0a64, screened nine fixed aiming/cadence variants on 24 new Center seeds. Eligible candidates needed at least 0.5 mean net-utility gain and no more than 0.5 mean kill loss

against rules. Only a one-window rest following observed ammo consumption qualified. It gained 0.750 utility while preserving mean kills at 6.5. Tighter-aim alternatives with higher utility failed the kill requirement.

Confirmation used 96 new seeds per scenario, the selected rule, original rules, a matched no-rest control, a command-rest control and five historical history fits, totaling 1,728 episodes. The selected policy gained 0.854 net utility over rules in Center (interval $[0.672, 1.050]$), with identical paired kills. It had identical gameplay outcomes to rules in Line: ammo replenishment hides expenditure in that scenario. Against the history-model bank, utility and kill criteria were not all established. The full gate failed. The confirmation does establish a narrow gain over rules in Center, without implying improved kills or per-shot accuracy.

6.2 Complementary observable events

A separate fixed candidate, frozen at 337629c, combines an observed ammo decrease with the preceding window's hit feedback. Either event triggers the same one-window rest. Ammo can identify a missed shot; hit feedback remains observable when replenishment masks expenditure. This controller reads no hidden weapon readiness and no future outcome.

The five development arms were combined events, rules, ammo only, hit only and command rest, each evaluated on 24 new seeds in both scenarios. The fixed combination qualified: utility gains were 0.792 in Center and 5.823 in Line, with identical kills. Confirmation then evaluated those five policies and five historical history fits on 96 fresh seeds per scenario, totaling 1,920 episodes.

The combination gained 0.823 net utility over rules in Center ($[0.643, 1.025]$), and 5.435 in Line ($[4.812, 6.070]$). Kills matched rules in all 192 paired episodes. Ammo-only matched the combination in Center and hit-only matched it in Line. However, comparisons against the history-model bank still failed the full gate: utility intervals included zero in both scenarios, and kill noninferiority was not established. A command-rest control remained stronger on Line. These results support an event-memory engineering baseline, not superiority over every control.

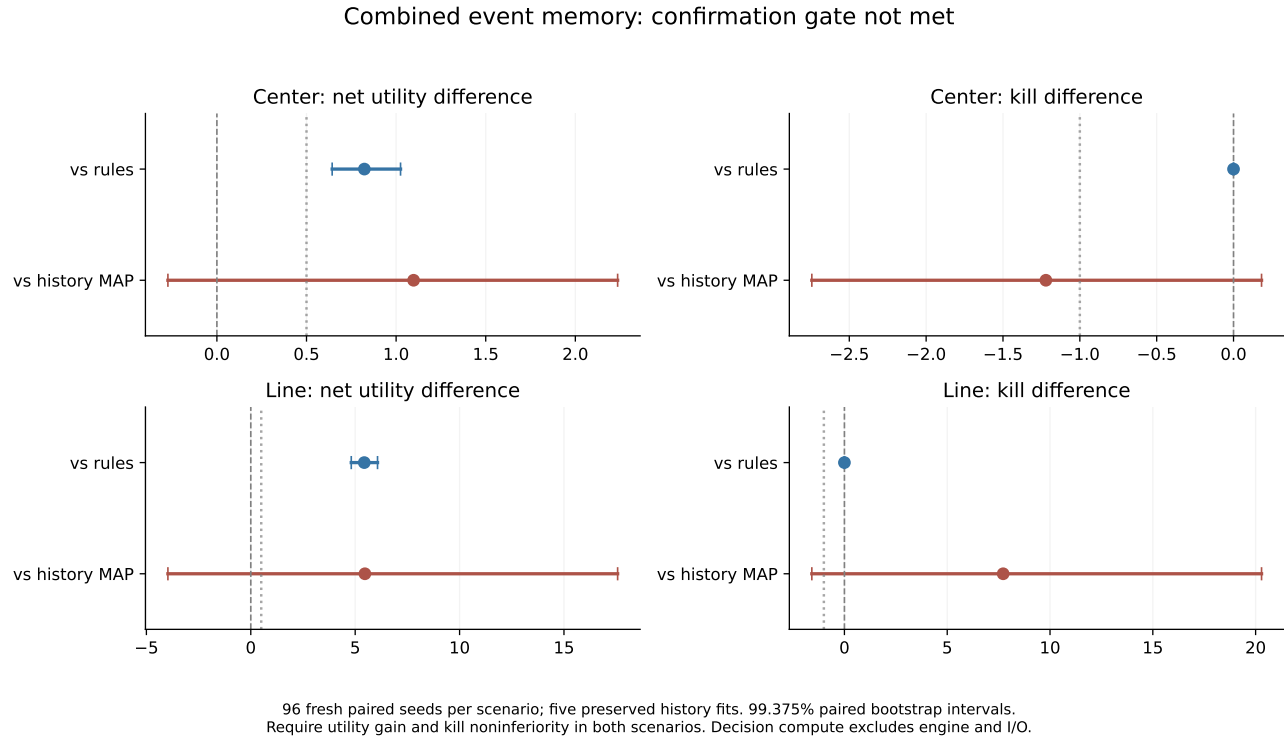


Figure 5: Combined observable-event memory: confirmed command-utility gains over rules, with unchanged paired kills, but a failed broader gate against the history-model bank. Dashed lines mark zero; dotted lines mark the 0.5 mean utility threshold or the -1 kill noninferiority boundary.

6.3 Matched ensembles and portable representations

The third study was frozen at 6402a29. Historical training was Center-only, and individual models transferred inconsistently to Line. The new current representation uses four features: bias, visibility, capped aim error divided by a target-width tolerance, and target width. It removes absolute ammo, health and distance. A history variant adds previous fire, previous hit, capped age since fire and a history-present bit, giving eight features. This changes both inputs and representation geometry, so it does not isolate a single variable as the mechanism.

All fitted ensembles reuse exactly the same 80 historical episodes, five training splits, issued-fire labels, Gaussian prior and optimizer. Each ensemble averages five MAP probabilities. Controls include an ensemble of the original 17-feature models, ruling out model averaging as an automatic explanation for representation-specific claims. All ensemble controls require a visible target; the unchanged historical single-model references retain their original behavior.

Eight arms with and without event memory were evaluated on 24 new seeds per scenario. Selection maximized the smaller of the two scenario gains among candidates passing the development utility and kill screens. The four-feature current-state ensemble with explicit event memory qualified, with development gains of 3.520 utility and 2.708 kills in Center, and 7.352 utility and 1.167 kills in Line. Learned-history variants were weaker on this criterion. These development results selected one candidate for confirmation.

Fresh confirmation ran 1,920 episodes, using 96 new seeds per scenario and ten arms counting the historical fits separately. Against rules, net-utility gains were 2.322 in Center ($[1.549, 3.129]$) and 5.663

in Line ($[3.590, 7.858]$). The full gate nevertheless failed. Center kill noninferiority against the historical bank was unresolved: difference 0.083, interval $[-1.440, 1.734]$. On Line, kill noninferiority against rules was also unresolved: -0.635 , interval $[-2.146, 0.875]$. The Line utility interval against the historical bank included zero. Both latency screens passed.

Secondary comparisons isolate the explicit event gate. Against the same fitted head without this gate, net-utility gains were 0.760 in Center ($[0.575, 0.961]$) and 4.846 in Line ($[4.236, 5.466]$), with kills identical in every one of the 192 paired episodes. These secondary intervals are exploratory and outside the eight-primary-comparison correction family. The gate avoids redundant issued-fire windows while preserving successful windows; this is reduced command cost, not established ammunition savings or improved combat performance. Against the original matched ensemble, the selected policy gained 3.271 utility in Center ($[2.386, 4.134]$), but averaged -2.228 on Line ($[-4.437, 0.015]$). The representation does not dominate across scenarios.

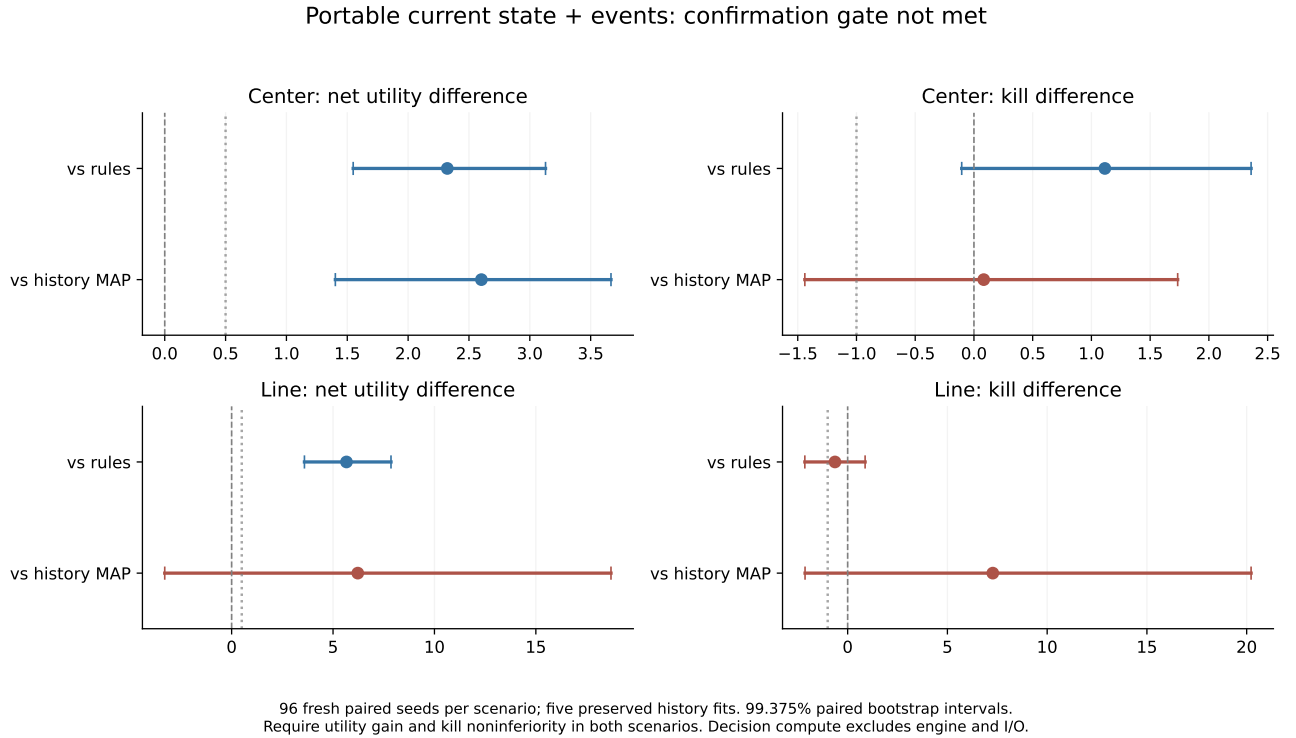


Figure 6: Fresh confirmation of the selected portable current-state ensemble with event memory. Utility gains over rules coexist with unresolved kill noninferiority and historical-bank comparisons. The full gate failed.

Taken together, the three prospective studies add 6,408 episodes and support a modest engineering baseline. None passes the broader continuation rule. Both scenarios were used in the last two development stages, so their confirmation is fresh-seed testing, not unseen-task validation. Reducing a costly but ineffectual fire command is useful under this utility definition; it does not by itself establish a general memory mechanism.

The relevant precedent includes learned action repetition and temporal abstraction (Lakshminarayanan et al., 2017; Sharma et al., 2017). Our cadence studies keep decision windows fixed and use simple observed feedback; they are not reproductions of those methods or a novel temporal abstraction algorithm. These studies establish no new Bayesian, recurrent, or language-model RL method and no ICLR novelty.

7 Separate gameplay and systems benchmarks

Original imitation benchmark. The shipped 5,253-parameter MLP uses 11 numeric features and two 64-unit hidden layers, with separate steering and firing heads. It was trained on 60,000 synthetic states labeled by an explicit rule controller. On 20 matched center-scenario seeds (1000–1019), mean kills were 17.55 for imitation, 17.60 for the teacher, and 0.95 for random control. These runs use two-tic actions and complete steering policies. Their kill counts must not be merged with or directly ranked against the seven-tic firing-only Bayesian study.

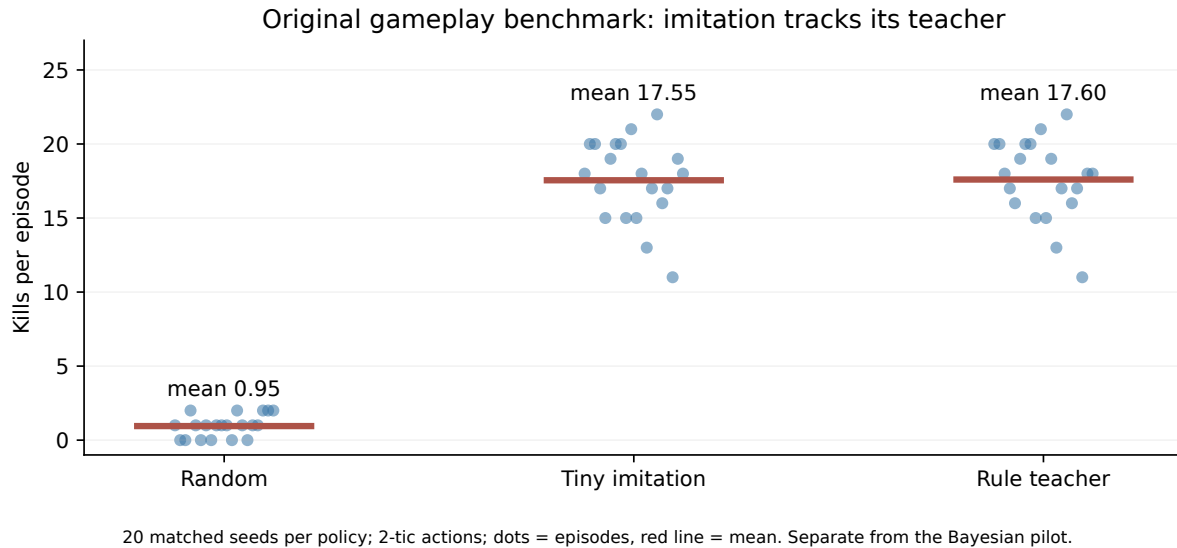


Figure 7: All 60 episodes of the original gameplay benchmark. The imitation policy approximates its teacher on this scenario. This is not a generalization or superiority claim.

Rust versus NumPy score kernel. Both implementations receive identical float64 logits with 32 rows, vocabulary width 151,936 and six candidates. They compute candidate probabilities, full-vocabulary candidate mass, and entropy. Inputs include extreme and equal-valued rows. After three warmups, each implementation runs 11 timed batches. Maximum output difference is 1.11×10^{-18} . Median elapsed time per row was 0.550013 ms for Python/NumPy and 0.203807 ms for Rust release, giving a ratio of 2.699.

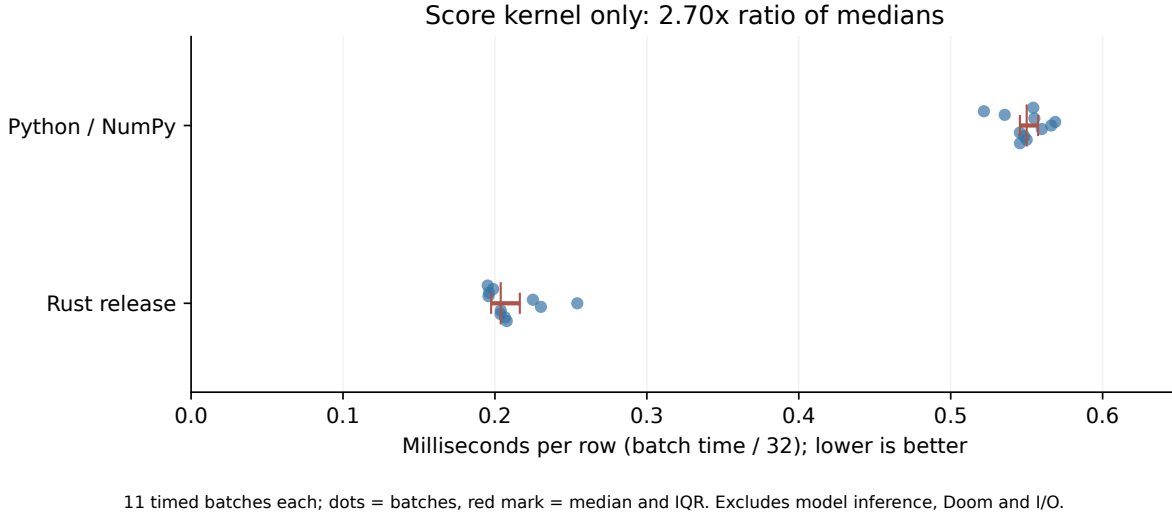


Figure 8: One local score-kernel microbenchmark: points are batch time divided by 32; red marks show medians and interquartile ranges, not confidence intervals. This excludes model inference, tokenization, Doom, HTTP, startup and I/O.

This measurement used Linux ARM64 in Docker with a two-CPU limit, Python 3.12.14, NumPy 2.5.3 and Rust 1.94.0. Rust ran before Python, and other Docker services remained active. NumPy already executes native numerical kernels, so the result compares these implementations, not interpreted arithmetic with compiled arithmetic. It is not evidence of a general Rust advantage or end-to-end agent acceleration. Randomized execution order, additional machines and end-to-end profiling remain necessary for stronger claims.

8 Related work and limitations

Calibration for learned dynamics and control is established research (Malik et al., 2019). The Laplace approximation used here is also established (Daxberger et al., 2021); applying it to this logistic head is not a new Bayesian method. RLCR combines correctness and calibration rewards for reasoning language models (Damani et al., 2025). We do not train or reproduce RLCR, and its results cannot be transferred to our firing-policy experiment.

The initial pilot’s linear features omit weapon cooldown and temporal state; the follow-up adds hand-crafted history but not a learned recurrent state. Its conditional-independence likelihood ignores within-episode correlation, which may invalidate the posterior’s uncertainty interpretation. The lower-bound policy may suppress useful firing, but this study does not isolate the mechanism. The evaluated studies use only two scenarios from one game and five training sets. The initial pilot uses ten evaluation seeds per scenario, and the memory follow-up uses thirty; this remains limited evidence for generality. Neither the eight disclosed language development questions nor a single gameplay GIF is an independent language benchmark. No evaluated connectome-inspired architecture, latent transformer, recurrent world model, semantic-entropy method or conformal decision guarantee is established by this report.

9 Continuation decision and reproducibility

Conditional on a future separately justified protocol, a follow-up could ask whether additional observations or recurrent computation improve the *decision*, conditional on their cost. Forecast learning and action selection should remain separately measurable. A proposed controller would allocate extra work only when estimated decision improvement exceeds its cost, rather than using posterior width alone. This is a research hypothesis, not an implemented contribution in these results. All full continuation gates in this report failed, so adaptive-compute efficacy work remains deferred. The event-memory command-efficiency gain is retained as a stronger engineering baseline.

Required controls include the same head without Bayes, fixed extra computation, scalar probability and entropy gates, and matched-data/compute ensembles. Random audit actions need known propensities and support checks. Development and confirmation splits should be fixed before tuning, with a second environment and an English decision task. Literature review must establish a distinct method or theoretical result before any ICLR novelty claim.

The repository retains the v1 and v2 execution snapshots at commits `4007e64` and `f59ca5c`, frozen protocols, source hashes, episode summaries, posterior weights and losslessly compressed traces. Hashes of raw traces refer to decompressed bytes. V2 completed in 85.18 workstation seconds; this duration is not a portable throughput benchmark. All five fits had gradient norms below 8×10^{-8} and positive covariance eigenvalues, validating numerical mechanics rather than model specification.

The memory run completed 1,980 new episodes in 412.13 workstation seconds. Its preserved analysis records the failed continuation gate. The browser demo uses pinned Qwen3-0.6B weights through WebLLM and runs separately from these numerical controllers; it is not a live Doom port or a quality benchmark. The historical Python package remains `openjev` for compatibility.

The plotting scripts regenerate figures and tables from preserved JSON, separately from experiment execution. Source and artifact hashes accompany the runs. Invalid v1 observations are not pooled into valid studies. The OpenJev repository retains historical artifacts and compatibility commands. The three later studies preserve compressed-trace byte hashes in their own manifests; their 6,408 new episodes are distinct from the original memory run. Their within-study adjustments do not correct the entire adaptive research sequence.

References

- Mehul Damani, Isha Puri, Stewart Slocum, Idan Shenfeld, Leshem Choshen, Yoon Kim, and Jacob Andreas. Beyond binary rewards: Training LMs to reason about their uncertainty, 2025. URL <https://arxiv.org/abs/2507.16806>. arXiv:2507.16806, version 2 revised May 2026.
- Erik Daxberger, Agustinus Kristiadi, Alexander Immer, Runa Eschenhagen, Matthias Bauer, and Philipp Hennig. Laplace redux: Effortless bayesian deep learning. In *Advances in Neural Information Processing Systems*, 2021. URL <https://arxiv.org/abs/2106.14806>.
- Michał Kempka, Marek Wydmuch, Grzegorz Runc, Jakub Toczek, and Wojciech Jaśkowski. ViZ-Doom: A doom-based ai research platform for visual reinforcement learning. In *IEEE Conference on Computational Intelligence and Games*, 2016. URL <https://arxiv.org/abs/1605.02097>.
- Aravind Lakshminarayanan, Sahil Sharma, and Balaraman Ravindran. Dynamic action repetition for deep reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 31, 2017. URL <https://ojs.aaai.org/index.php/AAAI/article/view/10918>.

Ali Malik, Volodymyr Kuleshov, Jiaming Song, Danny Nemer, Harlan Seymour, and Stefano Ermon. Calibrated model-based deep reinforcement learning. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97, pages 4314–4323. PMLR, 2019. URL <https://proceedings.mlr.press/v97/malik19a.html>.

Sahil Sharma, Aravind Srinivas, and Balaraman Ravindran. Learning to repeat: Fine grained action repetition for deep reinforcement learning, 2017. URL <https://arxiv.org/abs/1702.06054>. arXiv:1702.06054.